AuraSoft API Test Tool with an AI layer

System overview: architecture, what is built, how it works, and what is next

Status	**Phases 0, 1 and 2 complete**, now in Python · Phase 3 next
Date	24 October 2026
Repository	github.com/levan-petrosiani/Aurasoft-Monitoring · branch `ai-layer-python` (pull request to `main`)
Stack	Python 3.11+ · Flask · PostgreSQL 17 + pgvector · Claude (Anthropic) · Ollama (local)
Context	Hackathon project, AltaSoft

Contents

1. Summary
2. The problem and the approach
3. System architecture
4. How a test run works
5. Data store and privacy
6. The AI layer
7. Feature: AI run summaries (Phase 1)
8. Feature: standards → rules → human review (Phase 2)
9. Portal and command line
10. Quality: tests, verification, bugs found
11. Current state in numbers
12. What is left: roadmap and open items
13. Key decisions
14. How to run it

1. Summary

The **API Test Tool** (`aqt`) checks whether a bank's open-banking API (XS2A, Berlin Group NextGenPSD2 as profiled for Georgia) behaves the way the standard requires. It acts like a real third-party app (TPP): mutual TLS with a QWAC certificate, every request signed with a QSealC (`Digest` + `X-Jws-Signature`), OAuth2 with PKCE for customer authentication, and every response checked against the standard. Tests are YAML scenarios; banks are YAML profiles; a built-in **MockBank** lets everything run without a real bank. The tool was first built in C#/.NET; the team then moved it to **Python**, and the AI layer described here was ported with it (same database, prompts and checks).

This project adds an **AI layer** on top of that tool, built so that every AI output is traceable and checked:

- **Store everything** (Phase 0): every run, scenario, step, HTTP exchange and event goes to PostgreSQL, with tokens, authorization codes and IBANs masked before writing. Every model call is audited.
- **AI run summaries** (Phase 1): one click turns a run into a triage: problems grouped by cause, who is at fault (bank, our side, environment), evidence, clause, next step, and what changed since the bank's previous run.
- **Rules from the standards** (Phase 2): the Berlin Group guide and the Georgian guides are split into clauses and made searchable (also across English ↔ Georgian). The AI drafts *rules* from a clause, each with the exact sentence it rests on; code verifies every quote; a person approves each rule before it counts.

> **Design principle.** Facts are computed in code; judgement comes from the model; a person approves. Counts, run-to-run changes, scenario ids and clause references are computed or checked in code. The model groups, explains and drafts. Nothing the AI drafts becomes a rule or a test until a human approves it, and the test engine itself contains no AI: results stay reproducible.

Status at a glance

Phase	What it delivers	Status
0 · Foundations	PostgreSQL store, masking, import of existing runs, AI provider layer with audit log	done
Port to Python	Engine, CLI, MockBank, portal (teammates); store, standards, AI layer, AI pages and tests (this work)	done pull request
1 · Run summaries	AI triage of runs in CLI and portal; Claude or local model; validated in code	done
2 · Standards + rules	4 standards → 379 clauses, hybrid search, AI rule drafting, quote checks, review workflow, portal pages	done
3 · Scenario drafting	Approved rule → generated test scenario → validated → proven on MockBank → reviewed; coverage matrix	next
4 · Ask-the-agent	Staff ask questions about runs and the standards; agent answers with run ids and clauses	planned
5 · PIS	Payment initiation tests (designed in from the start)	planned
6 · Monitoring	Scheduled runs, regression alerts, trends	planned

93 automated tests, all passing	**379** clauses from 4 standards, all embedded	**24** rules drafted · 4 approved	**14** runs in the store · 19 AI summaries

2. The problem and the approach

The problem

- Writing conformance tests by hand from a 341-page standard (plus Georgian profiles) is slow, and it is hard to say which requirements are covered and which are not.
- Test results are long and technical; working out *why* a run failed, and whether it is the bank or our side, takes an expert.
- History lives in report folders; nobody can ask "when did this bank start failing?".

The approach — and why it is shaped this way

Original idea	What we built instead, and why
AI writes tests directly from the documents	**Document → approved rule → test.** A one-line rule with its quote can be reviewed in seconds; a generated test file cannot. Every future test traces back to a sentence of the standard.
AI "tests the APIs"	**The AI writes, the engine runs.** The model never calls a bank. Results are reproducible from YAML and auditable.
AI summaries	**Triage, not prose:** grouped by cause, bank vs our side, evidence, clause, next step; facts computed in code.
Rule drafting with human approval	Kept, and strengthened: the quote is **verified in code**, so a rule can never rest on words the standard does not contain.
Store every data	Kept, with **masking before writing** (tokens, codes, PSU ids, IBANs) and an audit log of every AI call.
RAG agent over logs	**An agent with query tools** over the database for logs (counts and time ranges need SQL, not similarity search); retrieval only for the standards text. (Phase 4)

Model choice: cloud and local

Drafting reads only public standards, so it uses **Claude** (best accuracy; `claude-opus-5-5`). Features that see run data (summaries, the future ask-agent) can be switched to a **local model** via Ollama so data never leaves the machine. Embeddings for search are always local (`bge-m3`). The default local chat model is `qwen3:8b` (5 GB): it summarized a run in 35 s versus 190 s for `gemma4:26b` (17 GB) with the same root cause found.

3. System architecture

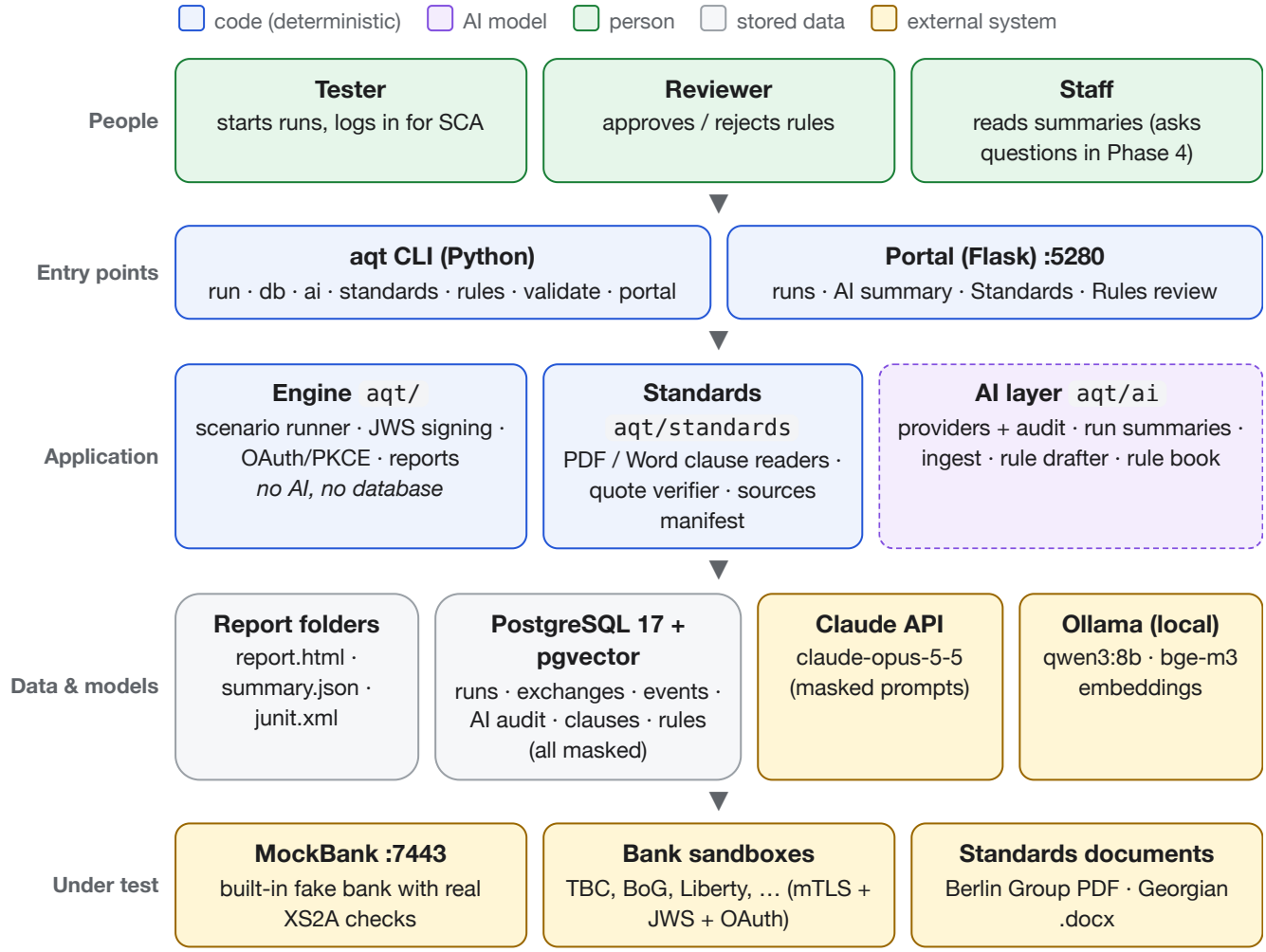


Figure 1 — Components. The engine has no AI and no database dependency; everything that reaches the database or a model is masked first.

Packages

Package	Responsibility	Lines
`aqt/` (engine)	YAML loading and binding, scenario runner, HTTP/TLS/SCA steps, JWS signing and verification, OAuth/PKCE, reports (HTML, JSON, JUnit)	3 281
`aqt/cli.py`, `cli_ai.py`	`aqt` command: run, list, validate, mock, portal · db, ai, standards, rules	818
`aqt/mockbank/`	Fake bank with real XS2A checks (mTLS, signatures, consents, OAuth/PKCE)	1 158
`aqt/portal/`	Flask web UI: dashboard, runs, AI summary panel, Standards, Rules review (Python + templates)	1 803
`aqt/store/` ported	PostgreSQL schema (SQL migrations) and access, masking, run import, clause and rule queries	1 027
`aqt/standards/` ported	PDF and Word clause readers, quote verifier, sources manifest (no AI, no DB)	452

`aqt/ai/` ported	Providers + audit, run summaries, standards ingest, rule drafting and review; prompts as files	1 239
`tests/`	pytest: engine, MockBank end to end, store, standards, AI, portal routes	1 212

Technology

Runtime	Python 3.11+ (developed on 3.14); Flask portal; PyYAML, cryptography, pyOpenSSL
Database	PostgreSQL 17 with pgvector (Docker, port 5435); psycopg 3 with plain SQL; numbered SQL migrations
AI	Official `anthropic` Python SDK (Claude); Ollama over HTTP for local models and bge-m3 embeddings
Documents	pdfplumber (PDF text), Office Open XML read directly (Word)
Install	Core: `pip install -e .[test]` ; store + AI layer is an optional extra: `pip install -e .[ai]`
Constraint	Standalone: only public PyPI packages; no dependency on AltaSoft's internal repositories

4. How a test run works

A run takes a bank profile and a set of scenarios. **Preflight** scenarios (certificates, TLS, OAuth discovery, a signed request) always run first; if a critical one fails, the rest are marked *Blocked* instead of failing one by one for the same reason. Each scenario is a list of steps; each request is signed, sent over mutual TLS, and its response is checked (status, error codes, headers, JSON fields, signature).

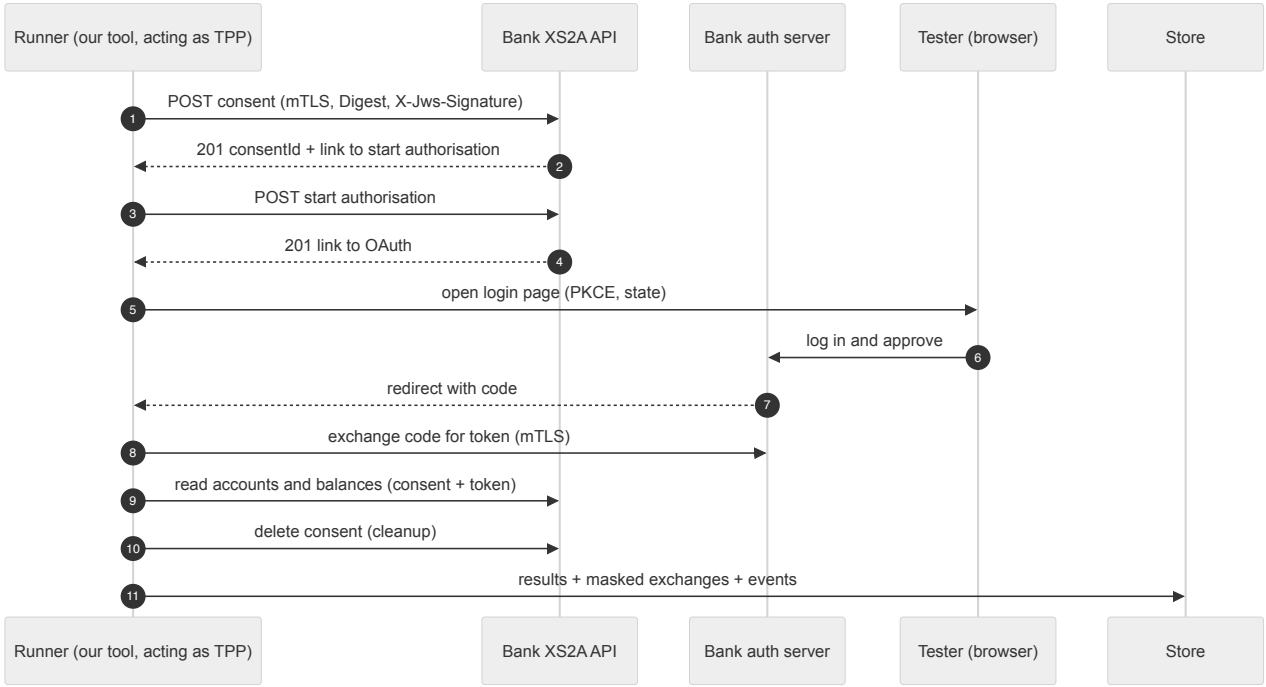


Figure 2 — An account-information scenario (AIS-001) from start to store.

Outcome	Meaning
Passed	All expectations met.
Failed	The bank's API did not behave as expected (a bank-side deviation).
Error	A problem on our side: configuration, certificates, timeout waiting for a login.
Blocked	Not run because a critical preflight check failed.
Skipped	The bank does not declare the feature (expected; not a problem).

Reports are written as before (`report.html` , `summary.json` , `junit.xml`), and the run is also saved to the database. A database problem never fails a run: it becomes a notice, and the folder can be imported later.

5. Data store and privacy

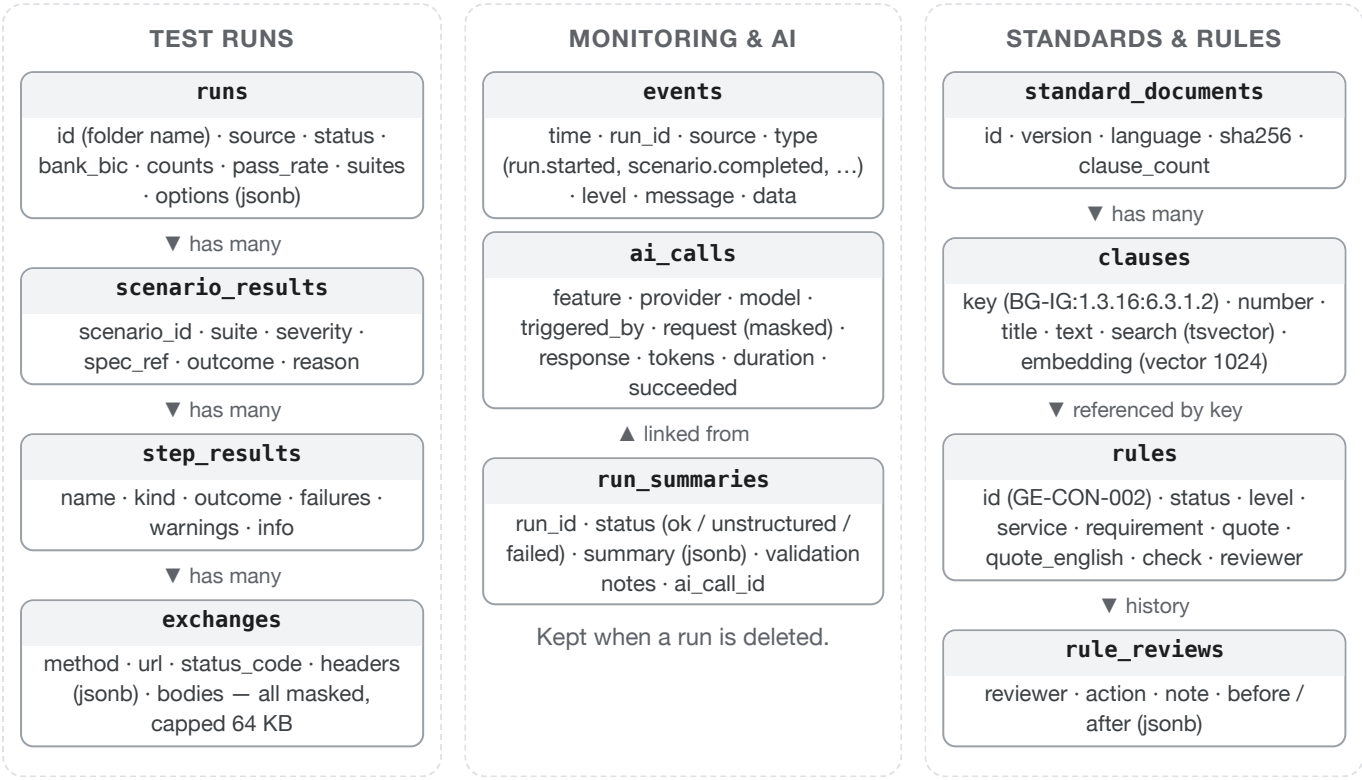


Figure 3 — Database tables (PostgreSQL), grouped by purpose; key columns only.

Masking: what never reaches the database or a model

Data	Treatment
Authorization, Cookie, PSU-ID headers	Value replaced (`Bearer ***(masked)`)
Tokens, codes, secrets in JSON, forms and URLs	`access_token` , `refresh_token` , `code_verifier` , … masked at any depth; the OAuth `code` in forms and query strings
XS2A error codes	**Kept readable** (`"code":"FORMAT_ERROR"` is evidence, not a secret) — a bug found and fixed in Phase 1
IBANs	Country, check digits and last 4 kept: `GE29**************4917`
Large bodies	Cut at 64 KB and flagged

> Verified on all 12 imported runs (139 exchanges): no tokens or codes in any form remain in the database. The same masker is applied to every prompt a model receives, as a second layer.

Audit

Every model call, successful or not, is a row in `ai_calls` : feature, provider, model, who triggered it, the masked request, the answer, token counts, duration and a prompt hash. Rule decisions are rows in `rule_reviews` with the fields before and after each change.

6. The AI layer

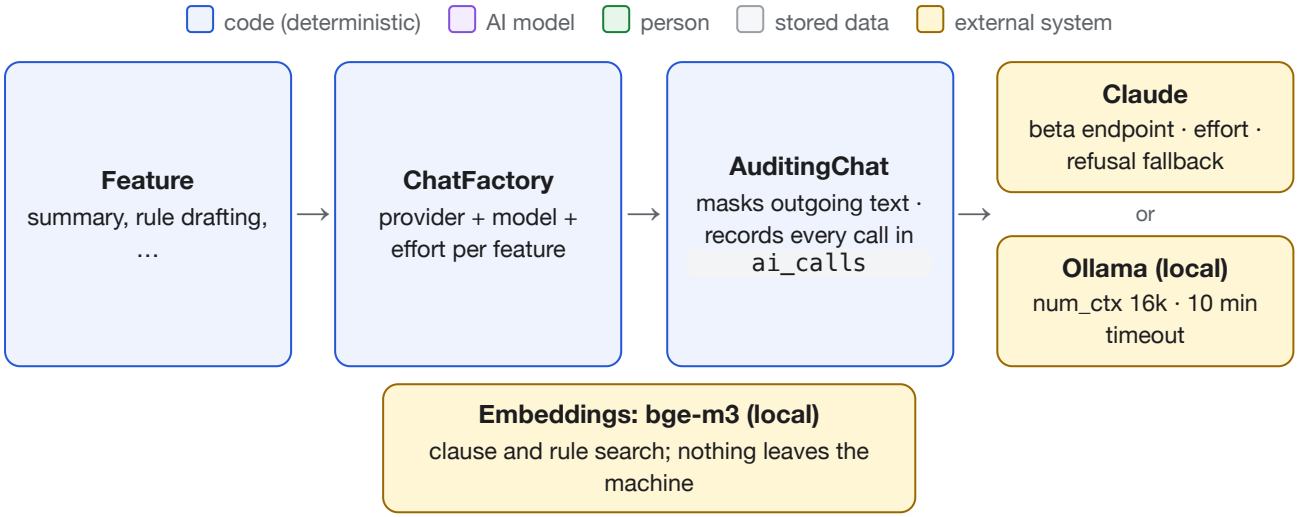


Figure 4 — Every model call goes through the same wrapper; features only see a common chat interface.

Feature	Default	Notes
Run summary	Claude, effort medium	Switchable to local (button in the portal, `--provider ollama`, `AQT_AI_SUMMARY_PROVIDER`)
Rule drafting	Claude, effort high	Reads only public standards; local model available for trials
Scenario drafting	Claude, effort high	Phase 3
Ask-the-agent	Claude, effort medium	Phase 4; switchable to local
Embeddings	bge-m3 (local)	Multilingual: English questions find Georgian clauses

Safeguards in the provider layer

- **Refusal fallback:** our prompts contain security-testing material (signatures, tokens, auth flows) that can trip a model's safety filters; a declined request is re-run on Anthropic's recommended fallback model instead of failing.
- **Context window:** Ollama silently cuts prompts longer than its default window; every local call now sets 16k tokens.
- **Timeouts** are recorded as failures (not as user cancellation), so they appear in the audit log.
- The exact request sent to Claude is checked by a test against a local fake Anthropic server (model, limits, effort, fallback header, streaming) — no key needed.
- Prompts and their JSON schemas are files in `aqt/ai/prompts/`, reviewed like code.

> **Not yet verified live:** calls to Claude. No `ANTHROPIC_API_KEY` has been set, so all live AI results so far come from the local `qwen3:8b`. The Claude request format is verified offline.

7. Feature: AI run summaries (Phase 1)

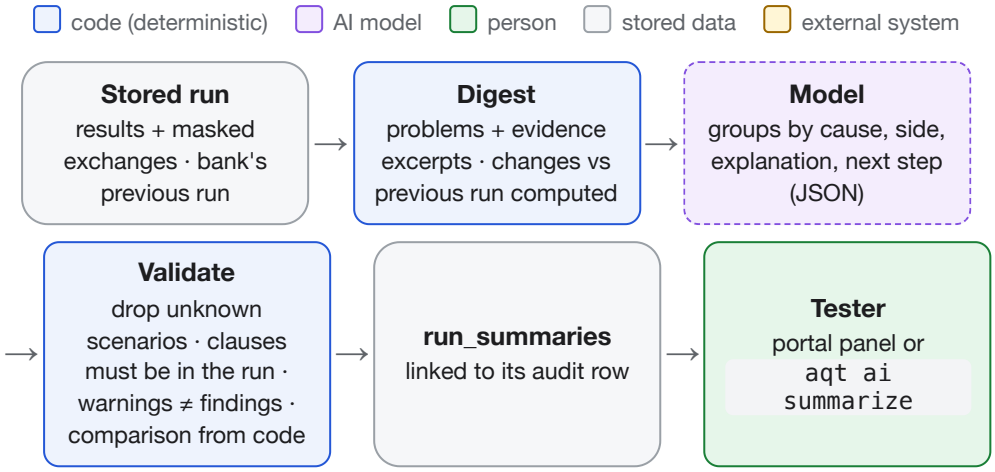


Figure 5 — From a stored run to a checked triage.

A summary contains: a **health** verdict (healthy / degraded / broken / inconclusive), **findings** grouped by root cause, each with the side at fault (bank, our side, environment, unclear), evidence quoted from the responses, the clause, and a next step; the change since the previous run; and warnings worth a look.

Evaluation on real data

All 9 stored runs with results were summarized with the local model and compared with the database. Five kinds of error were found and each was fixed — preferably in code:

Error found	Fix
"Since the previous run" described changes wrongly	Now written **in code** from the computed changes
Warnings of passed scenarios reported as problems	**Code** moves them to "worth a look"; findings about skipped scenarios are dropped (Python port)
Random 503s of the TBC test sandbox blamed on the bank	Prompt: temporary 5xx = environment unless persistent
A tester clicking "Deny" treated as a critical bank fault	Prompt: `access_denied` at login = unclear, suggest a re-run
A correct but shortened clause reference was removed	Validator matches clause numbers and restores the run's exact wording

> After the fixes, re-run on the 5 affected runs: all correct against the data. The real TBC deviation (global consent rejected with `FORMAT_ERROR` instead of `PARAMETER_NOT_SUPPORTED`) is found every time with the right clause. Local speed: 15–35 s per run.

8. Feature: standards → rules → human review (Phase 2)

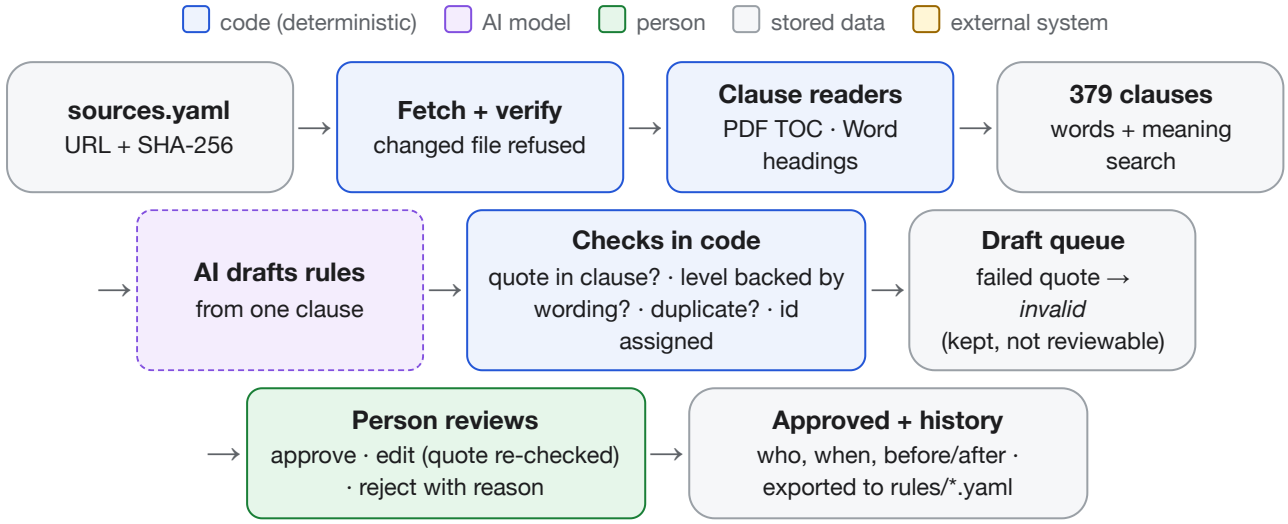


Figure 6 — The rule pipeline. Only a person moves a rule to approved.

The standards

ID	Document	Version	Language	Clauses
`BG-IG`	Berlin Group NextGenPSD2 XS2A Implementation Guidelines	1.3.16 (Dec 2025, latest)	English	182
`GE-XS2A`	Georgian XS2A Implementation Guide (openfinance.ge)	0.8.RC3	Georgian	133
`GE-JWS`	Georgian JSON Web Signature Guide	0.5.RC1	Georgian	32
`GE-IAA`	Georgian Identification, Authentication and Authorization Manual	0.8.RC1	Georgian	32

- **PDF:** the table of contents gives the clause list; each entry is found in order in the body; deeper sections (e.g. 6.3.1.2) are found inside their parent. Running headers and footers are dropped by pattern (pdfplumber puts footers last). A defect in the PDF (a stray sentence glued into the chapter 6 heading) is corrected in the manifest. The Python reader gives the same 182 clauses as the C# one; every existing rule quote still verifies.
- **Word:** heading styles give the structure; numbers are computed like Word. Verified against the published PDF of the Georgian guide, because the document's own cross-references contradict each other.
- The Georgian guides exist only in Georgian. The Georgian text is the source of truth; each rule carries an English translation of its quote, labelled as an AI translation.
- Documents are downloaded, not committed; a changed file (different SHA-256) is refused until reviewed.

What a rule looks like

```
GE-CON-002   MUST · AIS · GE-XS2A 9.1.1.10
Requirement  When validUntil is 9999-12-31, the bank sets the consent's validity to the maximum
             allowed (at most 90 days) and returns that date in the consent details.
Quote        "გამონაკლისს შეადგენს მნიშვნელობა „9999-12-31“, რომლის მიღების შემთხვევაში აუცილებელია
      ამსმპ-მა თანხმობის ვადა შეცვალოს მაქსიმალურ დასაშვებ მნიშვნელობაზე (არაუმეტეს 90 დღისა) ..."
EN (AI)      "The exception is the value '9999-12-31' ... must be replaced with the maximum
allowed
             duration (≤90 days) ..."
Check        positive — create consent with 9999-12-31, read it back, expect ≤ 90 days
Tool checks  ✓ quote found in the clause
```

Checks done in code before a person sees a draft

Check	Effect
Quote occurs in the clause	Ignores only spacing, line breaks, hyphenation, quote/dash style and case; "…" marks skipped words. Fails → rule stored as **invalid** (kept, never reviewable).
Level vs wording	MUST/SHOULD/MAY must be backed by the quote's keywords (English and Georgian); otherwise a warning for the reviewer.
Duplicates	Close in meaning (embeddings) **and** naming the same fields, codes and statuses; otherwise "accept availableAccounts" and "accept allPsd2" looked identical.
Review	Approve only drafts; reject needs a reason; an edited quote is checked again; a stale update is refused; every action recorded with the reviewer's name.

Findings from the first drafts (local model). It finds requirements and quotes them exactly, including Georgian — it found the Georgia-specific **90-day consent limit** (`PERIOD_INVALID`), which the Berlin Group guide does not have. Its weak point is the level: it marks optional functions as MUST; the level check flags this to the reviewer. In clause 4.1 it joined non-adjacent bullet items without "…": the quote check correctly rejected 8 such drafts (the prompt now forbids it). On GE-XS2A 9.2.2 it changed one Georgian word inside a quote (სათაურის → სათავის), and the check caught that too.

9. Portal and command line

Portal (Flask, http://localhost:5280)

Page	What it does
Dashboard	All bank profiles with last result and pass rate; MockBank start/stop
New run / Run	Start a run, watch it live, SCA login button; per scenario steps and HTTP exchanges; **AI summary** panel with "Summarize with Claude" / "Local"
Runs, Scenarios	Run history; scenario catalogue
Standards	Documents, outline with rule counts, search (English or Georgian, by words and meaning), clause view, "Draft rules with Claude / Local"
Rules	Review queue by status and standard; quote, translation, tool checks; approve / edit (incl. area, service, expected codes) / reject / reopen; reviewer name; "How to review" guide; export to `rules/`

Command line (`aqt`)

Command	Purpose
`run --bank X [--suite ..] [--with-mock] [--sca-auto]`	Run scenarios; saved to the store when `AQT_DB` is set
`db migrate | import | status`	Database schema, import of report folders, row counts
`ai check` · `ai summarize <run|latest>`	Provider check; AI triage of a run
`standards fetch | ingest | list | search | show`	Download and verify, split and store, browse and search clauses
`rules draft | list | show | approve | reject | reopen | export`	Draft rules from a clause; review; export approved rules
`validate` · `list` · `mock` · `portal [--db | --no-db]`	Check all YAML; list banks/scenarios; run MockBank; run the portal

10. Quality: tests, verification, bugs found

Test file	Tests	Covers
Engine (4 files)	22	YAML, variables, JSON paths, signing, reports, every shipped YAML, MockBank end to end (teammates' port)
`test_store.py`	20	Masking, run mapping, import, PostgreSQL round trips, search, a database created by the C# tool
`test_standards.py`	18	TOC parsing, page furniture, sub-sections, Word numbering, quote verifier (English and Georgian), manifest
`test_ai.py`	29	Options, exact Claude request (fake server), audit and masking, summaries, drafting, level and duplicate checks, review workflow, export
`test_portal_ai.py`	4	Pages with and without the database, reviewer name, edit and approve, redirect safety

93 pass with `AQT_TEST_DB` (throw-away database per test module); without it 76 pass and 17 are skipped.

> **Port verified against the C# results:** the Python store maps all 12 imported runs (139 exchanges) exactly as the C# tool stored them; the Georgian readers give identical clauses; the Python digest equals the C# digest; the existing database is used unchanged.

Bugs found by testing against real data, and fixed

Bug	Effect if left
Run crashed when the database was down	A storage problem would fail a test run
Masker hid XS2A error codes	The key evidence of failures unreadable to people and the AI
Engine's partial token masking (12 characters kept)	Token prefixes stored; now fully masked
Local model timeout at 100 s, not audited	Silent failures with large models
Ollama truncating long prompts silently	Rules drafted from half a clause
Full-text search required every word	Questions found nothing
Contents heading not counted in Georgian numbering	Every Georgian clause number off by one
Clause links redirected in a loop	Standards page unusable from links
False duplicates from embeddings	Different rules flagged as the same
Unstyled input boxes	Portal looked unfinished
OAuth callback port not reusable on macOS (Python port)	Second login scenario of every run failed on a Mac (15/16)
pdfplumber footer order (Python port)	No Berlin Group headings found
Georgian "optional" read as "mandatory"	Wrong level warnings on Georgian rules

All-digit SHA-256 read as a number	Manifest crash for such a hash

Every step, decision and finding is recorded in `NOTE.md` (42 entries); `ARCHITECTURE.md` holds the current diagrams.

11. Current state in numbers

Item	Value
Runs in the store	14 (MockBank, TBC sandbox, TBC test bank), 136 scenario results, 199 HTTP exchanges
AI run summaries	19 (local model)
Standards / clauses	4 documents, 379 clauses, all with embeddings
Rules	**4 approved** (BG-AUTH-003, BG-AUTH-004, BG-GEN-001, BG-SIG-002) · **11 drafts** to review · **9 invalid** (quote not verbatim)
AI calls audited	27, all to the local model (no Claude key yet)
Automated tests	93, all passing
Git	`main` : teammates' Python port. `ai-layer-python` : the ported AI layer, as a pull request

Approved rules so far

Rule	Level	Requirement
BG-AUTH-003	MUST	The bank provides dedicated authorisation endpoints for payment initiation.
BG-AUTH-004	MUST	The bank creates authorisation sub-resources when authorisation is started via the signing-baskets endpoint.
BG-GEN-001	MUST	Content parameters in the HTTP body are JSON or XML; XML only for ISO 20022 payment or account messages.
BG-SIG-002	MAY	If the signing-baskets endpoint is offered, the bank lets the TPP submit payment data without starting authorisation.

Note: three of the four approved rules concern payments (PIS) and signing baskets, which the tool does not test yet (Phase 5); BG-SIG-002 is tagged area SIG although it is about signing baskets (SBK) — the edit form can change it now.

12. What is left: roadmap and open items

Immediately

Item	Why
Review and merge the pull request	The Python AI layer is on branch `ai-layer-python`; it includes a small fix to `aqt/oauth.py` (macOS port reuse) that the teammates should see.
Set `ANTHROPIC_API_KEY`	No live Claude call has been made. Compare Claude's rule drafts and summaries with the local model on the same inputs.
Finish the review queue	11 drafts waiting: reject BG-CON-001..003 (request formats, not bank duties), edit then approve GE-CON-001, approve GE-CON-002, judge the 4.1 drafts with the review guide.
Re-draft clause BG 4.1	8 drafts failed because the model joined non-adjacent bullet items without "…"; the prompt now forbids it — draft the clause again.
Approve AIS rules	Phase 3 needs approved rules for what the tool can test today (consents, accounts) — e.g. the 90-day consent rules.

Phase 3 — scenarios from approved rules next

- **ScenarioDrafter:** from an approved rule, the AI writes a YAML scenario using the existing templates; then a loop in code: `validate` → dry run against MockBank → errors fed back (a few attempts).
- New scenario field `rules: [...]`; `aqt validate` warns about scenarios citing unapproved rules and approved rules without a scenario.
- **Drafts review page** for generated scenarios (diff-style YAML view, MockBank result) — approved scenarios go to `scenarios/` in git; drafts never run against real banks.
- **Coverage matrix:** clause → rules → scenarios → last result per bank.
- **MockBank gaps:** when an approved rule is not enforced by MockBank (e.g. the 90-day limit), that is a task, so generated tests can be proven before they meet a real bank.

Phase 4 — ask-the-agent planned

- A chat page and `aqt ask`: "Why did TBC fail last night?", "Since when does bank X return 503?", "Which rules cover consent expiry?".
- Read-only tools over the database (search runs, scenario history, exchanges, compare runs) and over the standards (clause search, rules). Answers cite run ids and clauses. A database role with read access to views only.
- An evaluation set of about 30 real staff questions with known answers, before staff use it.

Phase 5 — payment initiation (PIS) planned

- Payment templates and a `pis` suite: initiate → authorise (SCA) → status → cancellation; negative cases.
- A `poll` option for request steps (wait for a final transaction status).
- MockBank payment endpoints with a status lifecycle. Already designed in: `service: AIS | PIS | COMMON`.

Phase 6 — monitoring planned

- Scheduled runs per bank, alerts when a bank that passed starts failing, trend views.

- Retention: exchanges kept 90 days (configurable); results and summaries kept.

Open items and risks

Item	Detail
before staff use Login and roles	The portal has no login; the reviewer name is typed in. Real staff review needs authentication and an "approver" role.
Data policy for run data	Masked run data goes to Claude when summaries use Claude. Confirm this is acceptable for real bank data, or set summaries to local.
Real bank sandboxes	Most of the 16 bank profiles are placeholders; real certificates and credentials are needed. AIS has only run against MockBank and TBC.
English Georgian guides	Only Georgian versions exist; translations are AI-made per rule. Ask obstandards@association.ge for English versions.
Local environment	On this Mac, uv-created venvs mark the install file hidden and Python 3.14 skips it: run `python -m aqt` from the repository root.
Not verified by clicking	Portal actions are covered by route tests and screenshots, not by browser automation; auto-summary after a run is untested.
Continuous integration	No CI pipeline yet (pytest with a PostgreSQL service on each push).
Retention	Designed (90 days for exchanges) but not implemented.

13. Key decisions

Decision	Choice and reason
Language	Python (team decision; the teammates ported the engine, verified both ways against the C# client and MockBank). The AI layer was ported to match.
Database	PostgreSQL (Docker) with pgvector: one database for runs, audit, clauses and vectors. Python uses plain SQL on the same schema; the database the C# tool created is kept.
Models	Claude for drafting (public documents, best accuracy); local Ollama option for features that see run data; local embeddings.
Local model	`qwen3:8b` by default (5× faster than `gemma4:26b`, same root causes found); bigger models selectable.
Standards	Latest versions: Berlin Group IG 1.3.16; Georgian guides from openfinance.ge (Georgian only).
Translation	No whole-document translation; Georgian text is the source of truth; per-rule English translation labelled as AI.
Rules storage	Database is the source of truth (queue, history, search); approved rules exported to YAML for git.
Approval	The team approves now; real staff later (needs login). Every action recorded with name, time and reason.
Scope	AIS first; PIS designed in from the start, built in Phase 5.
Logs Q&A	Agent with database tools, not vector search over logs.
Repository	Public GitHub repository; source documents not committed (licences); no secrets committed.

14. How to run it

```
python -m venv .venv && source .venv/bin/activate
pip install -e ".[test,ai]"
docker compose up -d                                    # PostgreSQL + pgvector on
localhost:5435
aqt db migrate
aqt db import reports

# a full test run against the built-in fake bank, saved to the database
aqt run --bank MOCK --with-mock --sca-auto --db
"Host=localhost;Port=5435;Database=aqt;Username=aqt;Password=aqt_dev"

# AI (local model needs Ollama with qwen3:8b and bge-m3; Claude needs ANTHROPIC_API_KEY)
aqt ai check
aqt ai summarize latest --provider ollama

# standards and rules
aqt standards fetch && aqt standards ingest
aqt rules draft GE-XS2A:0.8.RC3:9.1.1.10 --provider ollama

# web portal (store/AI pages on when the database is reachable)
aqt portal                                              # http://localhost:5280

# tests (including database tests)
AQT_TEST_DB="host=localhost port=5435 user=aqt password=aqt_dev" pytest
```

Further reading in the repository

`README.md`	User manual: commands, profiles, scenarios
`ARCHITECTURE.md`	Current diagrams (updated after every phase)
`docs/ai/PLAN.md`	AI plan, phases and decisions
`docs/ai/REVIEW-GUIDE.md`	How to approve, edit and reject rules
`NOTE.md`	Journal: every step with what and why
`standards/sources.yaml`	Standards: versions, URLs, hashes